The dataset and the TUG test

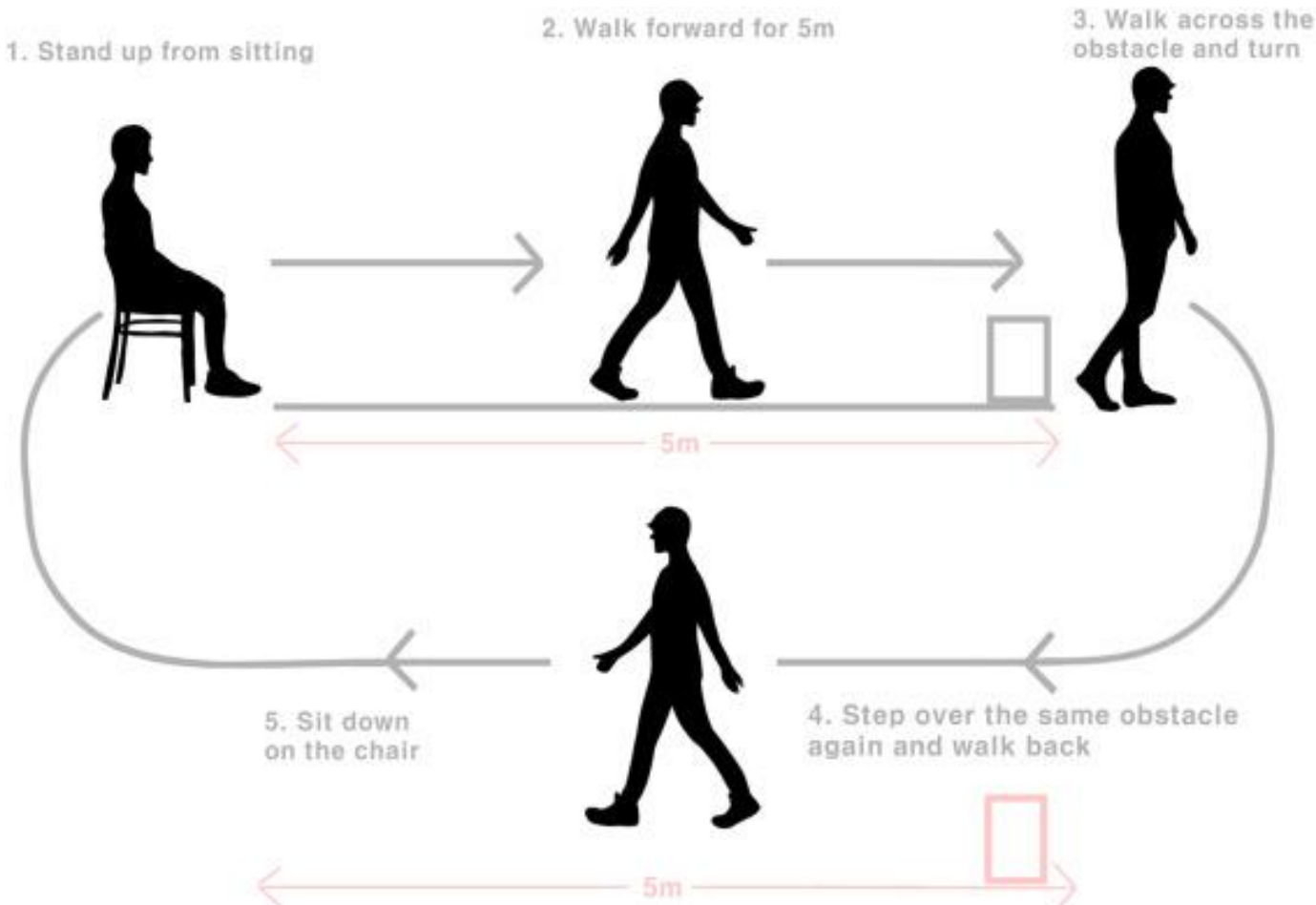
Symbols and acronyms

Every symbol and acronym used in these slides, with its unit. Each is also explained where it first appears.

symbol	meaning	unit
ω	angular rate measured by the gyroscope	°/s
b	gyroscope bias: a slowly varying offset of the rate	°/s
θ, ϕ, ψ	angles: pitch (or tilt), roll, yaw (rotation about the vertical)	°
f	specific force read by the accelerometer: all forces but gravity, per unit mass	m/s^2
g, a	gravity (9.81 m/s^2); acceleration of the device	m/s^2
R	rotation matrix: the device axes written in the reference axes	—
q	unit quaternion: axis and angle of a rotation in 4 numbers	—
Δt, f_s	sampling step; sampling rate $f_s = 1/\Delta t$	s, Hz

acronym	meaning
IMU	inertial measurement unit
MEMS	micro-electro-mechanical system
ADC, LSB	analogue-to-digital converter; value of one step
MCU, MMU	microcontroller; memory management unit
RTOS	real-time operating system
TDMA	time-division multiple access
RBS, TPSN, FTSP	synchronisation protocols for sensor networks
GNSS	satellite positioning (GPS and others)

The test


1. Stand up from sitting
2. Walk forward for 5m
3. Walk across the obstacle and turn
5m
4. Step over the same obstacle again and walk back
5. Sit down on the chair
5m

Why the Timed Up and Go

A clinical test of basic mobility: from a chair, stand up, walk 3 m, turn, walk back and sit down, while an observer measures the time. It can be used to evaluate balance, agility, strength and gait patterns in a short test.



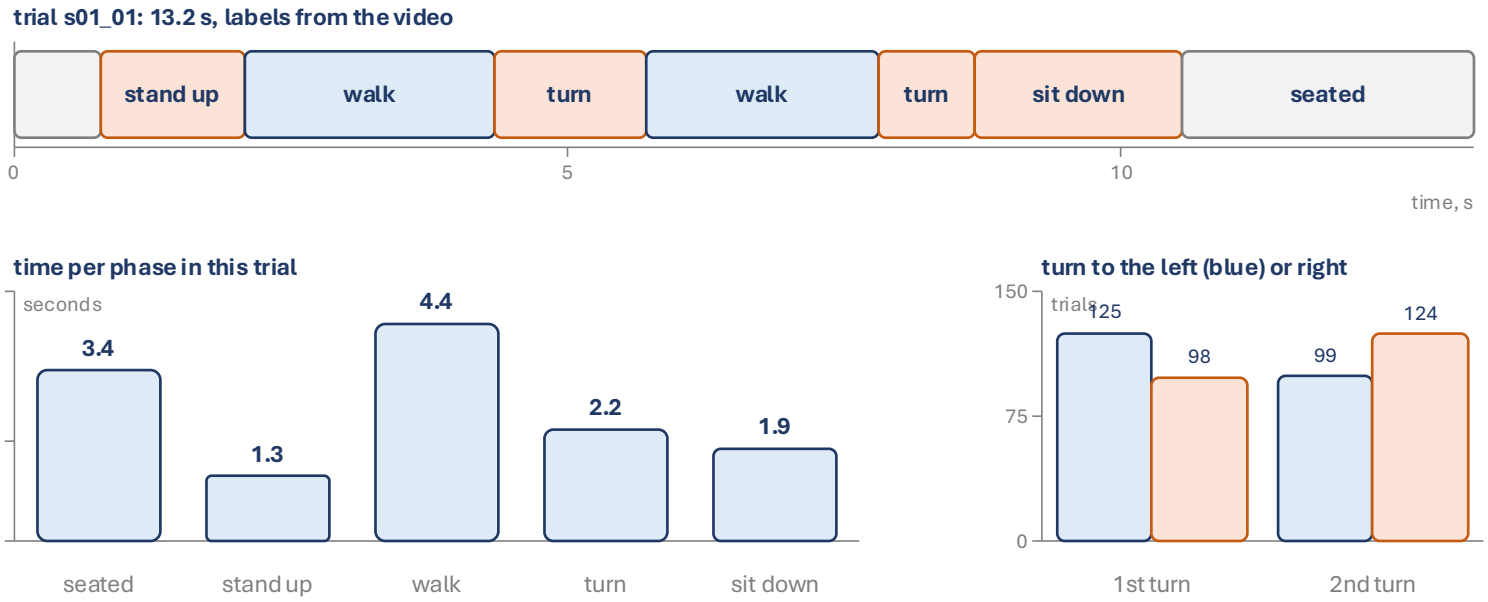
What it measures	Who it is for	Why automate it
basic mobility: balance, agility, strength; the clinical result is the total time, in seconds	older adults, for fall risk: 13.5 s or more marks a higher risk in community-dwelling older adults [2]; people with Parkinson's disease, where results are associated with severity; also healthy adults and children	a trained observer is needed for every execution; sensors allow repeated tests at home and the duration of each phase, not only the total

It is cheap and quick, but needs an expert with a stopwatch, and gives only the total time. Sensors can make it self-administered at home and measure each phase: the system described in the next slides is desgined to do exactly this.

[1] Podsiadlo and Richardson, J. Am. Geriatr. Soc. 39(2), 1991; [2] Shumway-Cook, Brauer and Woollacott, Phys. Ther. 80(9), 2000; [3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §I.

The TUG in the dataset

Each trial is one Timed Up and Go: from seated, stand up, walk 3 m, turn, walk back, turn and sit down.
The dataset holds 223 trials of 23 people, about ten each;
example for trial s01_01



the protocol
a standard chair and a line at 3 m; the person walks at a comfortable pace

the classes
five phases; seated appears at the start and at the end

durations
each phase has a different duration, meaning each of the classes is represented in different proportions in a trial

The phases are the main classes that an automated TUG system wants to predict: seated, standing up, walking, turning, sitting down.
They last from under a second to a few seconds, and the direction of the turns changes from trial to trial.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (executions_info.csv: first_turn, second_turn).

The dataset at a glance

Everything to know before opening the files: **who** was recorded, **with what**, **how often**, and what each column means.

the recording

source	Matey-Sanz et al., Data in Brief 51, 2023 · open on Zenodo
people	23 (s01 … s23), about 10 trials each
trials	223 Timed Up and Go → 446 CSV files, one per device
devices	phone in the front trouser pocket · watch on the left wrist
sensors	accelerometer + gyroscope, 3 axes each, asked at 100 Hz
labels	5 phases, one per sample, written from the video
side tables	subjects: age, sex, height, weight, hand trials: phone orientation, turn directions

one file: s01_01_sp.csv

column	unit	first row
`x_acc, y_acc, z_acc`	m/s^2	−4.16, −1.18, −8.99
`x_gyro, y_gyro, z_gyro`	rad/s	0.003, −0.014, −0.032
`timestamp`	ms since 1970, device clock	1657533977813
`label`	phase, from the video	SEATED

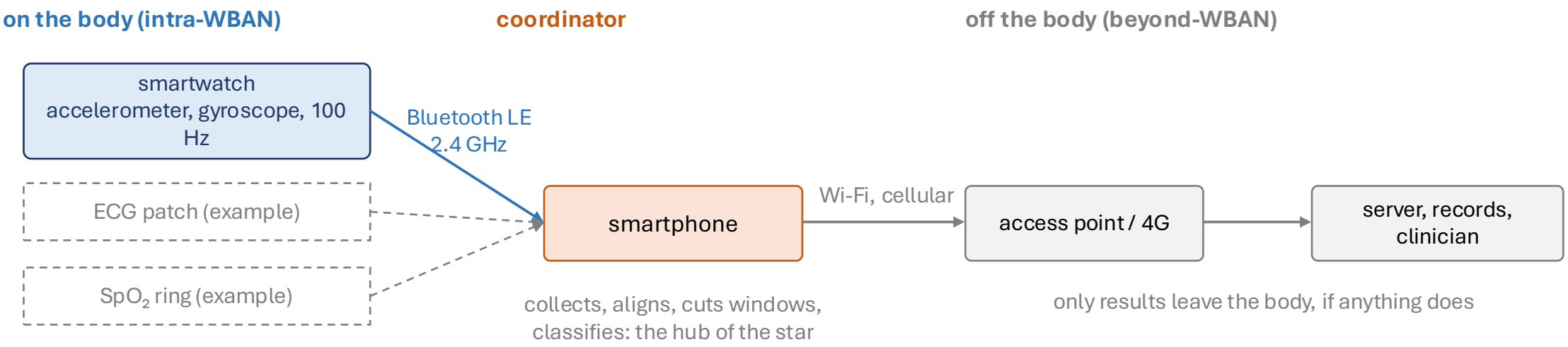
1351 rows in 13.2 s: about 100 per second, on average

Two sources of time: the **timestamps** come from each device's own clock, the **labels** from the video. The authors aligned the two with NTP.

[4] Matey-Sanz, Casteleyn, Granell, Data in Brief 51, 2023; Zenodo 8398688; trial s01_01, phone, first row.

A wireless body sensor network

A **wireless body sensor network** (WBSN), also called a wireless body area network (WBAN): small sensor nodes on the body, linked by short-range radio to one **coordinator** that relays the data off the body.



energy the smallest node sets the budget: 580 mAh in the watch, 5160 mAh in the phone

radio the body absorbs 2.4 GHz: the link depends on posture and on the side of the devices

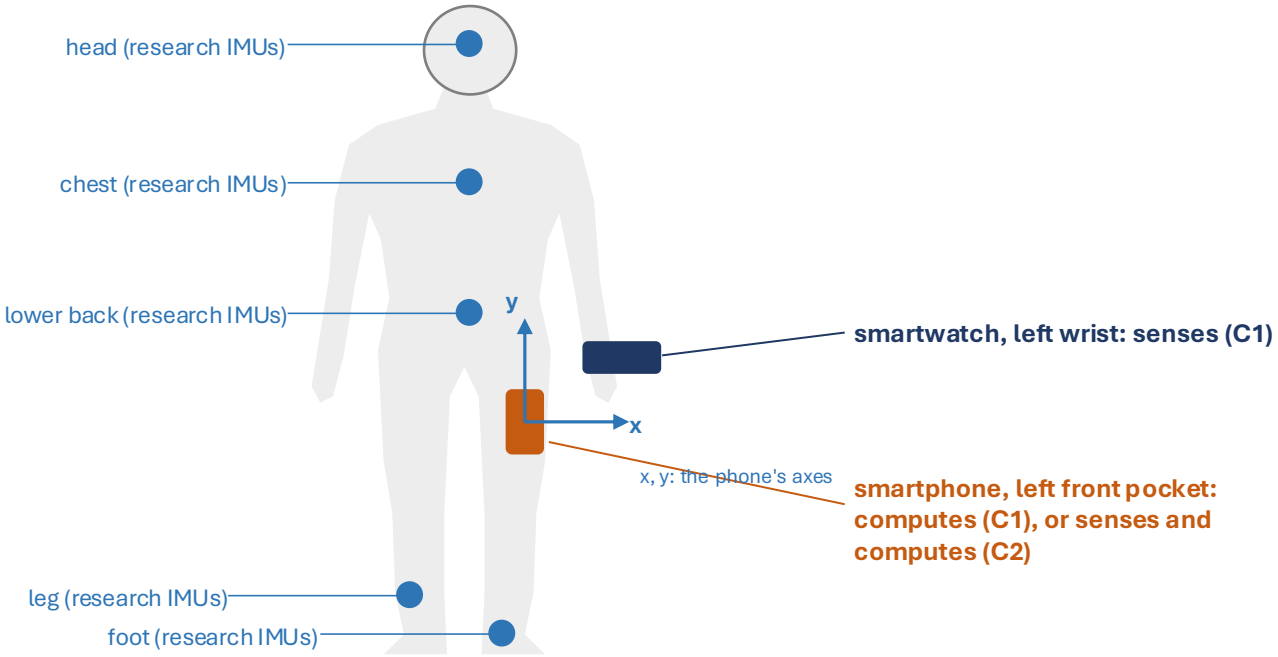
data health data: privacy and security on every hop

This system is the **smallest WBSN**: one sensor node, the watch, and one coordinator, the phone, in a star over Bluetooth instead of the WBAN standard IEEE 802.15.6.

[5] Hajar, Al-Kadri and Kalutarage, Computers & Security 104, 2021, (WBAN, also known as WBSN); [6] Chen et al., «Body area networks, a survey», Mobile Netw. Appl. 16(2), 2011; [7] IEEE Std 802.15.6-2012; [3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §II-B.

Gait analysis devices: lab and unsupervised settings

Typical gait analysis systems attach dedicated sensors, usually inertial measurement units (IMUs) to the lower back, chest, legs or head; this dataset instead tries to employ what a person already carries: a smartwatch on the left wrist and a smartphone in the left front trouser pocket.



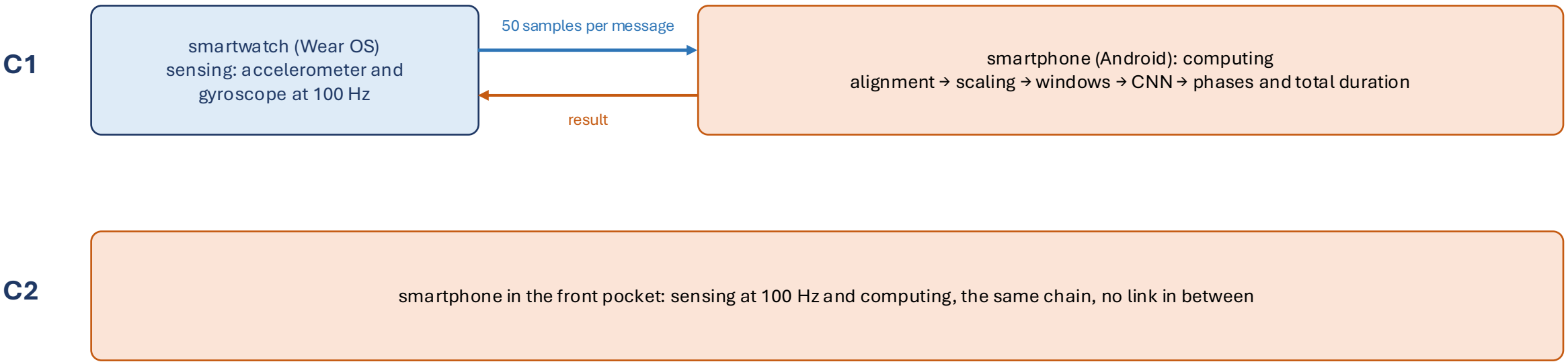
	dedicated IMUs	this system
devices	research sensors	a smartwatch and a smartphone
placed by	an expert, in a lab	the person autonomously
orientation	fixed by a strap, consistent across trials	free

Everyday positions make the test possible at home, at a price: the pocket's depth and the phone's orientation change from person to person (no instructions were given), so the same movement appears on different axes. Moreover, the devices are not placed by a professional or fixed.

[3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §I and §II-C2; [4] Matey-Sanz et al., Data in Brief 51, 2023, (executions_info.csv: orientation).

The TUG system: two configurations

The system from which the data was obtained automates the TUG with the same devices used to record the dataset. In C1 a Wear OS smartwatch senses and an Android smartphone computes; in C2 the smartphone does both.



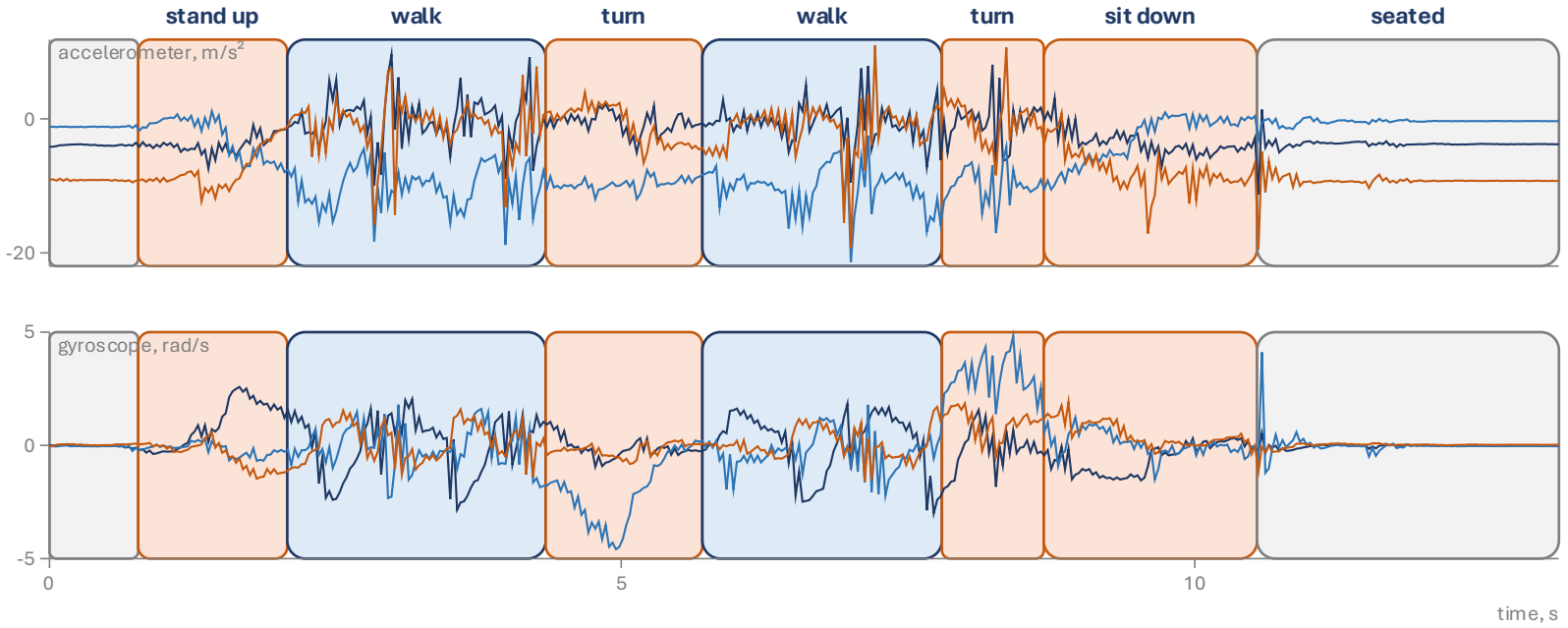
link: Bluetooth, through the Wear OS message API; the test ends by itself when the person is seated again

Where the decision is computed, and what crosses the link, are design choices: here the watch only senses and the phone computes. The next slides follow one sample through C1.

[3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §II-B, Fig. 1.

The six columns of one trial

The six columns of trial s01_01 over time, with the phases underneath: three axes of the accelerometer, in m/s^2, and three of the gyroscope, in rad/s. Each phase leaves a different pattern on the two sensors. An automated system learns to identify the phases based on these patterns.



x axis
dark blue line: across the thigh

y axis
blue line: along the thigh when standing

z axis
orange line: out of the screen

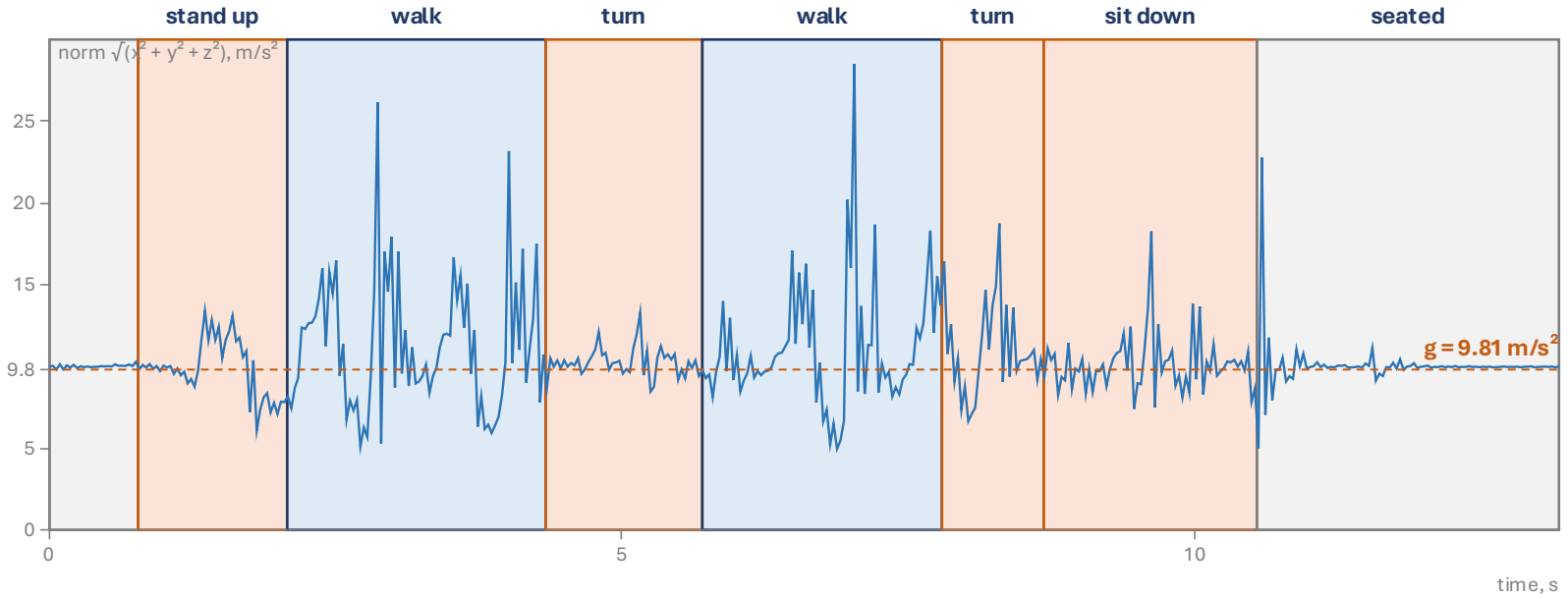
magnetometer
not in this dataset: no heading from the Earth's field

Seated: flat lines, gravity only. Standing up and sitting down: the accelerometer changes level because the phone tilts. Walking: oscillations at the step rhythm on both sensors. Turning: one gyroscope axis, y here, takes a large value.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone); resampled to 50 Hz.

What the accelerometer measures

An accelerometer measures specific force: gravity plus the acceleration of the device, along its own axes. The norm of the three axes does not depend on how the phone sits in the pocket.



phase	norm, m/s^2
seated	9.99 ± 0.63
stand up	9.87 ± 1.69
walk	11.18 ± 3.85
turn	10.45 ± 1.79
sit down	10.19 ± 2.23

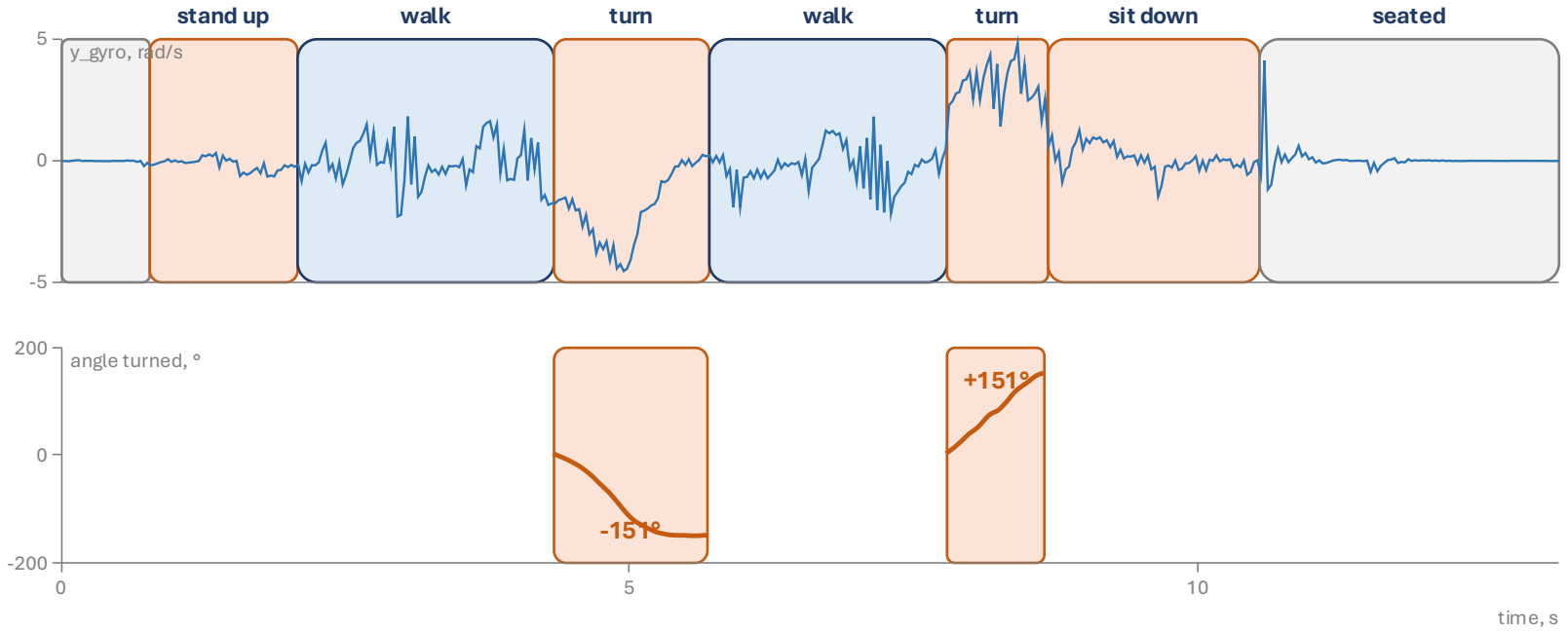
mean ± standard deviation of the norm in each phase of this trial

Seated, the norm is 9.99 ± 0.63 m/s^2: gravity, g = 9.81 m/s^2, and nothing else. Walking, it swings around g by ± 3.8 m/s^2: the movement adds to gravity, it never replaces it.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone); resampled to 50 Hz; specific force f = a − g.

What the gyroscope measures

A gyroscope measures angular rate, in rad/s, about the device's axes. Integrating the rate over time gives the angle turned: here the y axis during the two turns of the trial.



rate
how fast the device rotates, rad/s; 1 rad/s ≈ 57 °/s

angle
the integral of the rate over a turn

bias
a small constant error in the rate grows linearly in the angle. Integration only works in very short time.

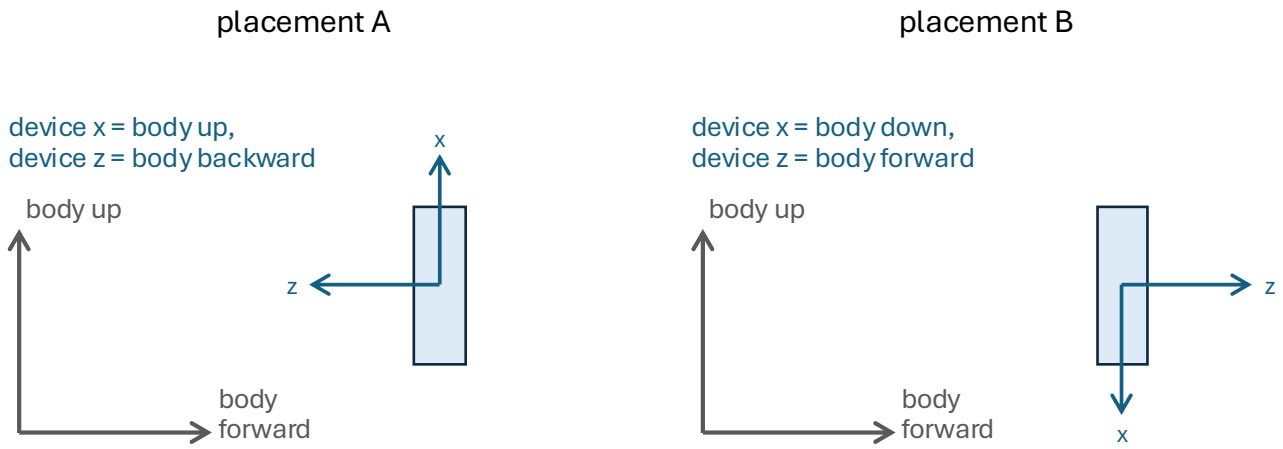
the turn
about 150°, not 180°: the phone turns with the thigh, not exactly with the body

The two turns give -151° and +151°: opposite signs, because the dataset records the first turn to the left and the second to the right. Integration also accumulates the gyroscope's bias: fine for a few seconds, not for minutes.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone); executions_info.csv (first_turn, second_turn); angle = ∫ ω dt, trapezoidal rule.

Device frame and body frame

The sensor reports along its own axes, whatever the body does. In the dataset the phone sat in a pocket **with no instruction about orientation**: 23 subjects, 4 distinct placements.



Drawing: the same phone in two placements.

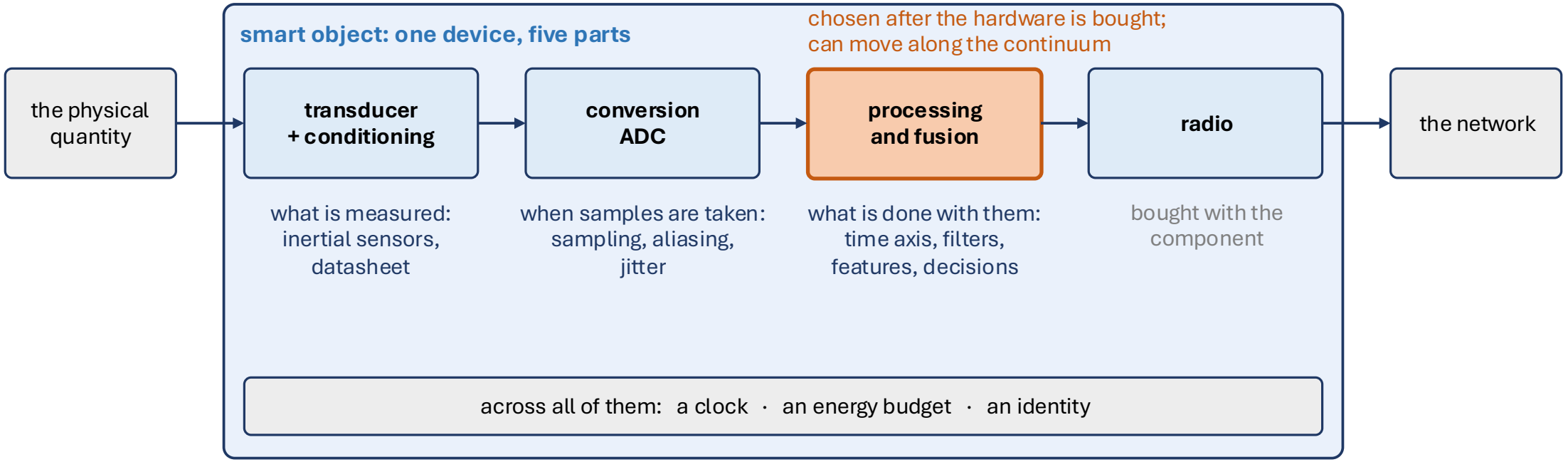
Feature computed on	std across the placements
one axis, x_acc	1.3 m/s^2
the norm ‖f‖	0, to machine precision

The norm is **invariant to the orientation of the device**, a single axis is not. It is not invariant to where on the body the device sits, and it discards the direction. Per-axis features degrade on an orientation never seen in training, and this shows only in an evaluation on held-out subjects.

Matey-Sanz et al., IEEE J. Biomed. Health Inform. 28(11), 2024; spread computed on a synthetic window.

Smart object: functional blocks

One device, five parts: the drawing names them and says what each one is responsible for.



Processing, the third part, matters most here: it is the only one the designer still chooses after the hardware is bought, and the only one that can be moved elsewhere: to the phone, a gateway or the cloud.

RFC 7228

RFC 7228 sorts constrained devices by **memory**, RAM for data and flash for code, not by processor or radio: memory decides which protocol stacks, security and software fit.

	Class 0 (C0)	Class 1 (C1)	Class 2 (C2)	beyond C2
RAM (data)	≪ 10 KiB	≈ 10 KiB	≈ 50 KiB	≫ 50 KiB
flash (code)	≪ 100 KiB	≈ 100 KiB	≈ 250 KiB	≫ 250 KiB
on the network	no secure direct Internet link: through a gateway or proxy	CoAP over UDP and DTLS, without a gateway; no full HTTP + TLS	the stacks of notebooks and servers (HTTP, TLS)	protocols are no longer the limit
operating system *	bare metal	an RTOS can be fitted	an RTOS	Linux, Android
if compromised *	low impact	medium	medium to high	—

C0 reaches the Internet only **through a gateway**; C1 speaks **constrained protocols** (CoAP, DTLS) on its own; C2 runs the usual stacks. Values are indicative; rows marked * are not in the RFC.

*[1] RFC 7228, §3 and Table 1; [3] rows marked * from Ojo et al., IEEE Access 6, 2018, not part of the RFC.*

Devices: CPU, memory, network

Six current devices, from low-end to high-end: most of them go well beyond the classes of RFC 7228.

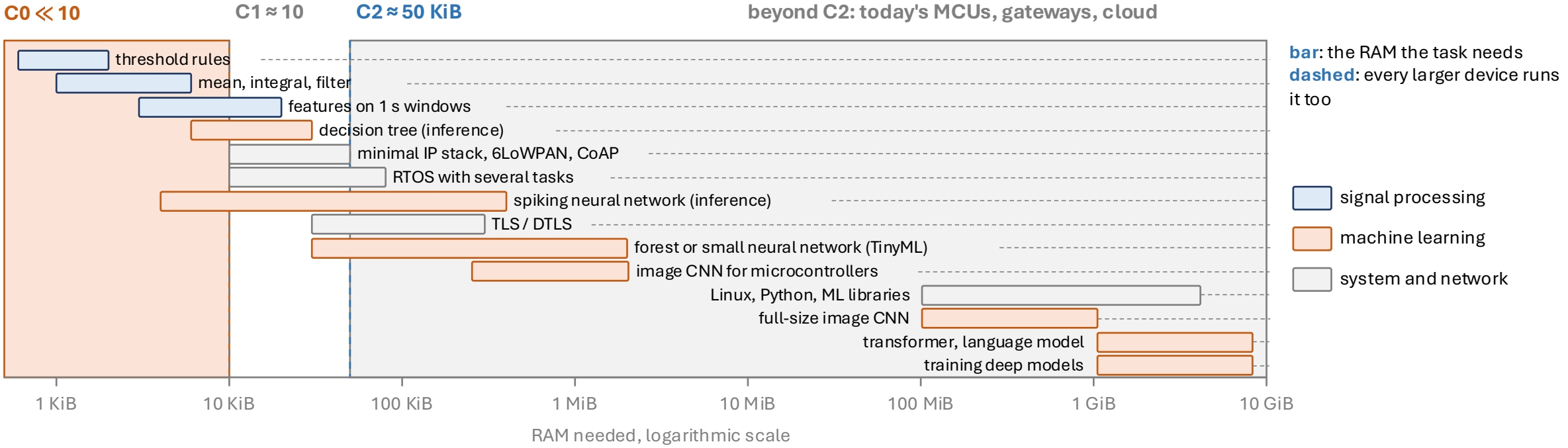
	CPU	RAM	code / storage	radio	software	RFC 7228
Arduino Uno · low-end	8-bit AVR, 16 MHz	2 KiB	32 KiB flash	none	bare metal	C0
nRF52840 · low-end	Cortex-M4F, 64 MHz	256 KiB	1 MiB flash	BLE, 802.15.4	bare metal, RTOS	beyond C2
RP2040 · middle-end	2 × Cortex-M0+, 133 MHz	264 KiB	external flash	Wi-Fi, BLE (Pico W)	bare metal, RTOS	beyond C2
ESP32-S3 · middle-end	2 × Xtensa LX7, 240 MHz	512 KiB	external flash	Wi-Fi, BLE	FreeRTOS	beyond C2
Orange Pi One · high-end	4 × Cortex-A7, ~1.2 GHz	512 MiB	microSD	Ethernet	Linux	—
Raspberry Pi 5 · high-end	4 × Cortex-A76, 2.4 GHz	up to 8 GiB	microSD, NVMe	Wi-Fi, BLE, Ethernet	Linux	—

Only the 8-bit Arduino Uno is C0; today's 32-bit microcontrollers are beyond C2 although most still run bare metal or an RTOS. Single-board computers (such as the Raspberry Pi) and phones run a full operating system, Linux or Android, and act as gateways and edge nodes.

[4] Microchip ATmega328P, Nordic nRF52840, Raspberry Pi RP2040, Espressif ESP32-S3, Allwinner H3, Raspberry Pi 5, (BCM2712); [5] tiers after Ojo et al., IEEE Access 6, 2018; [1] RFC 7228.

What each class can process

Memory decides which processing fits on a node. Each bar is a task, placed by the RAM it needs: from a threshold rule, a few bytes, to training a model.



Each task needs **at least** its RAM, and runs on anything larger: a gateway can run TinyML, a microcontroller cannot run a transformer. Going up: **vision**, **transformers**, and training.

[1] RFC 7228, (classes); [8] Lin et al., MCUNet, NeurIPS 2020, (ImageNet on a microcontroller with 320 KB of SRAM); RAM of the tasks: indicative orders of magnitude, not measurements.

Energy limits and power use

RFC 7228 also classifies devices by how their **energy** is limited (E) and how they use **power** to communicate (P): for a battery node, often the real constraint.

energy limitation: how long it can keep working

E0	event energy-limited: energy for each event	a button that powers its own message
E1	period energy-limited: energy within a period	battery recharged or replaced; solar node through the night
E2	lifetime energy-limited: energy for the whole life	a sealed primary battery: discarded when empty
E9	no direct quantitative limitation	mains-powered

power strategy: how quickly it can be reached

P0 normally-off	sleeps for long periods; reattaches to the network when it wakes up
P1 low-power	reduced power, still attached: appears connected, perhaps with high latency
P9 always-on	always on and reachable

A sealed-battery sensor (E2) is **normally-off** (P0) or **low-power** (P1); a mains-powered gateway (E9) can be **always-on** (P9).

[1] RFC 7228, Table 3 (energy limitation) and Table 4 (strategies of using power for communication).

MCU or application processor

The processing unit of a smart object is an embedded system: a computer dedicated to a few functions. Two families: the micro controller (MCU), everything on one chip, and the application processor (SoC), with a memory management unit.

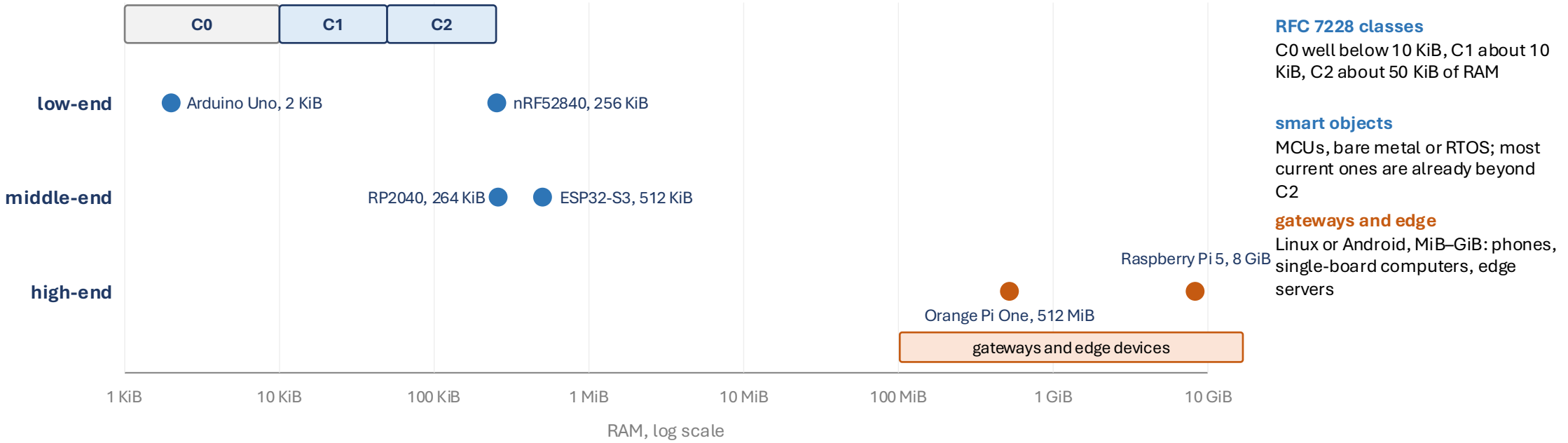
	microcontroller (MCU)	**application processor (SoC)**
core	Cortex-M, RISC-V, AVR	Cortex-A, x86
memory	RAM and flash on the chip, KiB–MiB; no MMU	external RAM and storage, MiB–GiB; with MMU
software	bare metal or RTOS	Linux, Android
power	µA asleep, mA active	hundreds of mW to W
example	Arduino Uno: 16 MHz, 32 KiB flash, 2 KiB RAM	Orange Pi One: 4 × Cortex-A7, 512 MiB

The difference that counts is the MMU: without it, bare metal or an RTOS and µA in sleep; with it, Linux or Android and hundreds of mW. Many current MCUs integrate a radio (ESP32, nRF52) and remain MCUs.

[8] Microchip ATmega328P and Allwinner H3 datasheets; [9] Ojo et al., IEEE Access 6, 2018.

Three tiers of devices

IoT devices span more than six orders of magnitude of RAM. Low-end and middle-end ones are MCUs, the smart objects; high-end ones are single-board computers and phones, which usually act as gateways or edge devices.



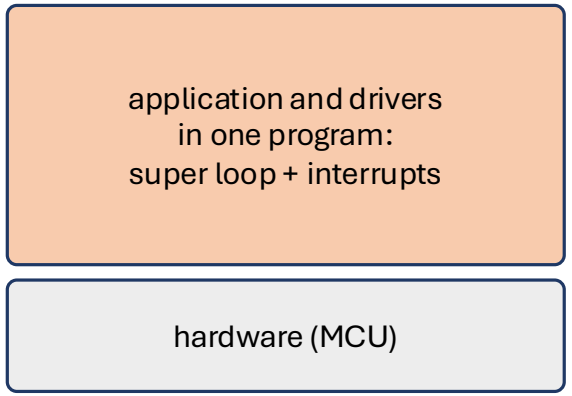
From the smallest to the largest: 6.6 orders of magnitude. RAM decides what a device can run: an IP stack, TLS, an operating system, a model. The RFC 7228 classes make this precise.

[10] RAM from the datasheets, (KB in the datasheets, KiB here); [11] tiers after Ojo et al., IEEE Access 6, 2018; [12] RFC 7228, §3.

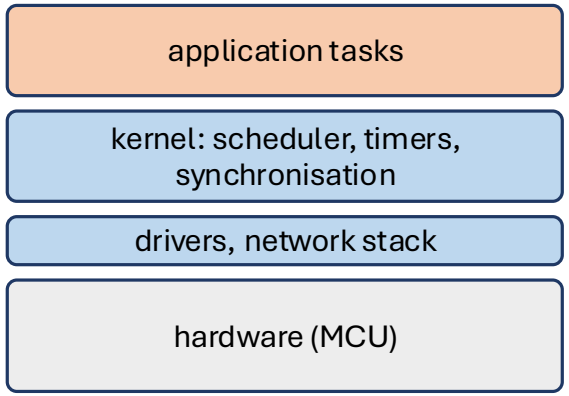
Bare metal or operating system

Software on a smart object runs either **bare metal**, one program that controls the hardware directly, or on an **operating system** that manages processor time, memory and peripherals for several tasks.

bare metal



with an operating system



super loop (bare metal)

```
while (1) {
    if (data_ready) read_sensor();
    if (window_full) compute();
    if (packet_ready) send();
    sleep_until_interrupt();
}
```

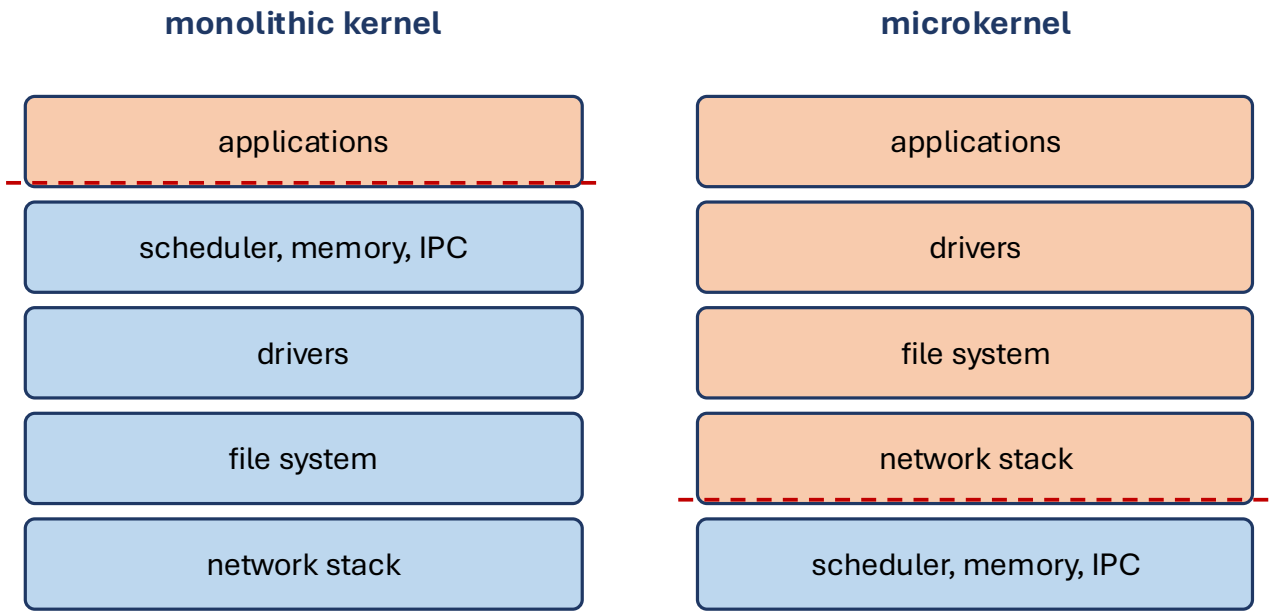
a long computation delays the next read:
the programmer must split it by hand

Bare metal fits a node with one simple job; an operating system pays off when the node must sample, compute and communicate at the same time, with deadlines and concurrent tasks.

[13] Farahani et al., IEEE Internet of Things Magazine, 2025.

Kernel architectures: monolithic and microkernel

The **kernel** is the part of the operating system that runs with full privileges. A **monolithic** kernel holds all services; a **microkernel** keeps only scheduling, memory and communication between tasks.



above the red line: user space · below: kernel space (orange: user, blue: kernel)

monolithic

all services in the kernel: fast calls,

a faulty driver can stop everything

e.g. Linux

microkernel

only scheduling, memory, IPC in the kernel; services are separate tasks:

isolated, but more messages

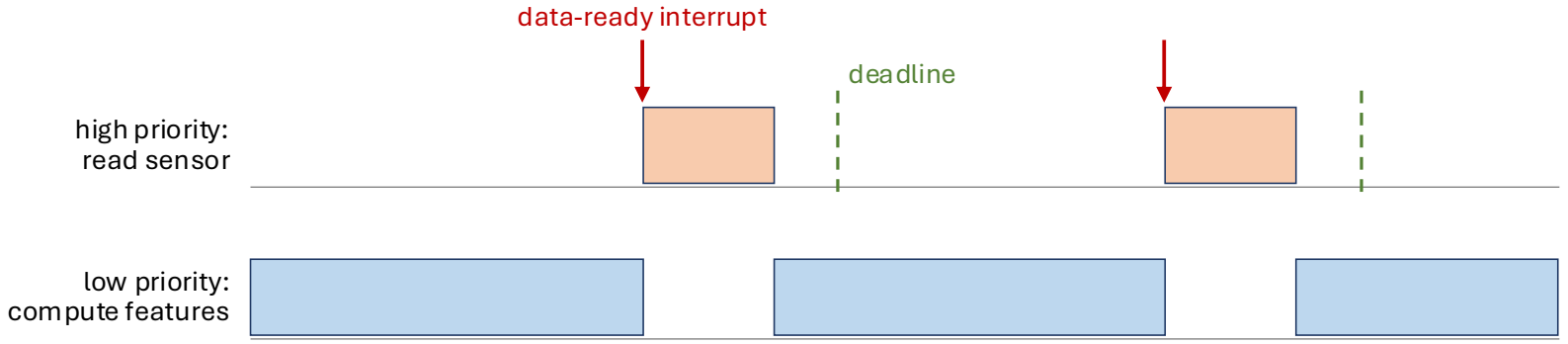
e.g. QNX, RIOT (micro-kernel design)

The trade-off is speed against isolation. On microcontrollers without an MMU the separation is weaker: kernel and tasks often share one address space, whatever the design.

[14] Tanenbaum and Bos, Modern Operating Systems, 4th ed. (2015), §1.7; [15] Farahani et al., IEEE IoT Magazine, 2025; [16] RIOT documentation.

Real-time operating systems

A **real-time operating system** (RTOS) guarantees that a task runs within a bounded time after its trigger: high-priority tasks **pre-empt** lower ones, so the worst-case latency is known in advance.



real-time = bounded latency

not «fast»: the worst case is known

hard: a missed deadline is a failure

soft: it degrades the result

programming models

event-driven: handlers on one stack,

very light (Contiki-NG)

multithread: one stack per task,

simpler to write (FreeRTOS, Zephyr)

example: samples at 50 Hz must be read every 20 ms; a late read shifts the sampling instant (see «Sampling-time error and amplitude error»)

On a sensor node the sampling task gets the highest priority: each read happens within its deadline even while the node computes or transmits.

[17] Buttazzo, Hard Real-Time Computing Systems, 3rd ed. (2011), Ch. 1–2; [18] FreeRTOS and Zephyr documentation; [19] Contiki-NG documentation.

Operating systems for smart objects

The operating systems most used on IoT devices, from bare metal to Linux, ordered by the resources they need.

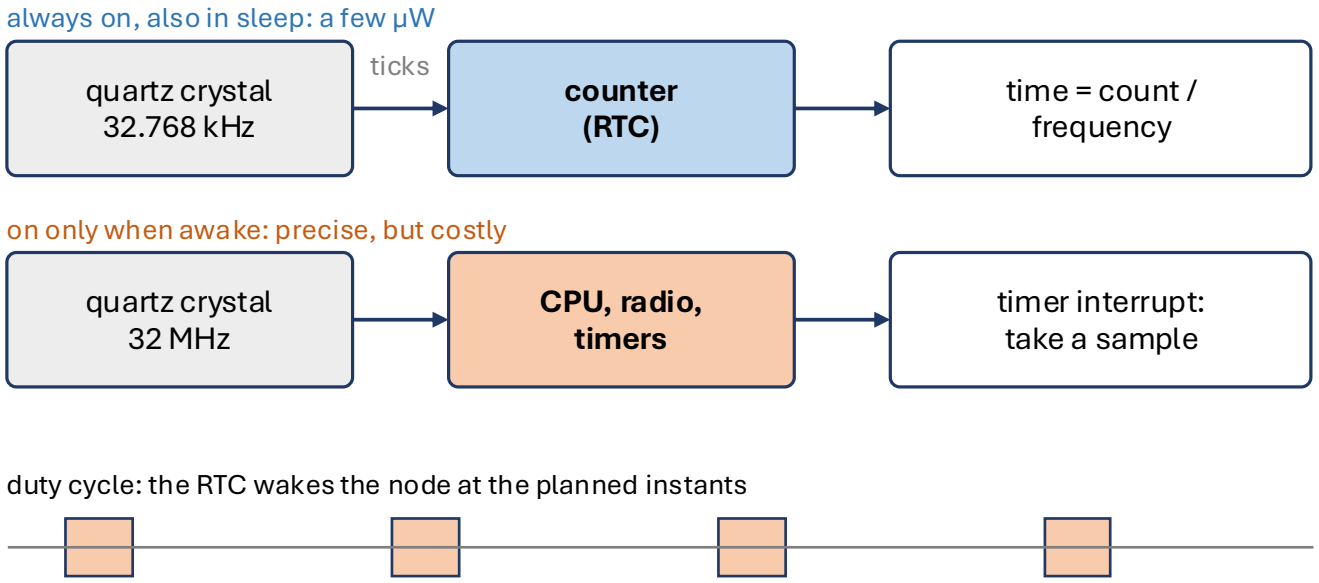
	kernel	programming model	typical device	RAM
bare metal	none	super loop + interrupts	class 0–1, single task	from a few kB
FreeRTOS	small real-time kernel	multithread, pre-emptive	MCU, class 1–2	kB
Zephyr	real-time kernel, configurable	multithread, pre-emptive	MCU, class 1–2	kB – tens of kB
RIOT	microkernel	multithread, real-time	MCU, class 1–2	kB
Contiki-NG	event-driven	protothreads (events)	MCU, class 1	kB
Linux (PREEMPT_RT)	monolithic	processes, threads	gateway, edge (MMU)	tens of MB +

Microcontroller nodes run small real-time kernels (FreeRTOS, Zephyr, RIOT, Contiki-NG); Linux, with the real-time patch, runs on gateways and edge devices with an MMU and megabytes of RAM.

[13] Farahani et al., IEEE Internet of Things Magazine, 2025; [12] RFC 7228, (device classes); [20] project documentation of each OS.

Where a node's time comes from

A node keeps time by counting the oscillations of a quartz crystal. Most nodes have two: a slow, low-power one that never stops, and a fast one used only while awake.



every tick is slightly off

a quartz is accurate to 20–50 ppm:

50 ppm = 4.3 s per day

it also changes with temperature

consequences

· samples taken at slightly wrong instants

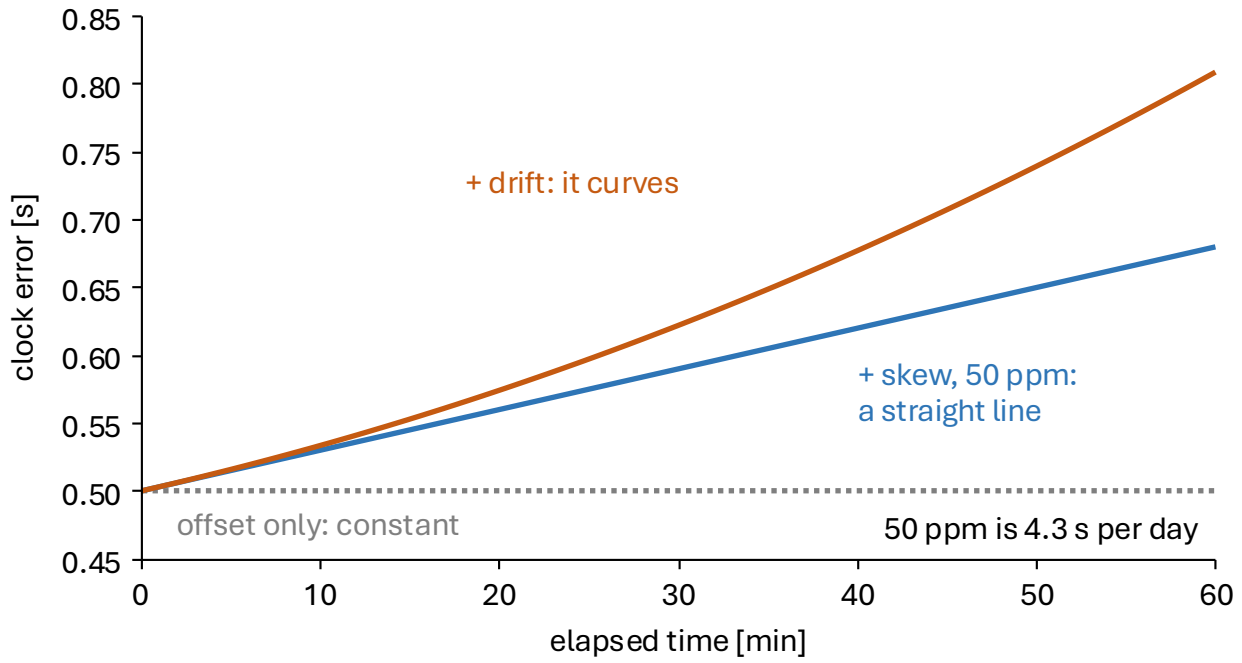
· two nodes drift apart: wake-ups need guard times, timestamps disagree

Sampling instants, timestamps and wake-ups all come from these oscillators, so their frequency error becomes timing error everywhere: the offset, skew and drift of the next slide.

[28] Epson FC-135 crystal datasheet (32.768 kHz, ±20 ppm at 25 °C); [29] nRF52 product specification, (32 MHz and 32.768 kHz clock sources).

Clocks: offset, skew and drift

Fusing nodes, ordering events, TDMA slots, agreement with a reference: all need **one time axis**. Each node's clock has error terms of different order: an offset becomes a **bias**, a skew a **trend**.



$$C(t) = C(t_0) + (1+\rho)(t-t_0) + \tfrac{1}{2}D(t-t_0)^2 + \varepsilon(t)$$

$C(t_0)$	offset arbitrary at power-up
ρ	skew, frequency error quartz: 20–50 ppm
D	drift: ageing, temperature ppm per year, ppm per °C
$\varepsilon(t)$	phase noise short-term jitter

Aligning the offset (**synchronisation**) and the frequency (**syntonisation**) are different: correcting only the first leaves the clocks diverging at a rate ρ.

Sources of timestamp error

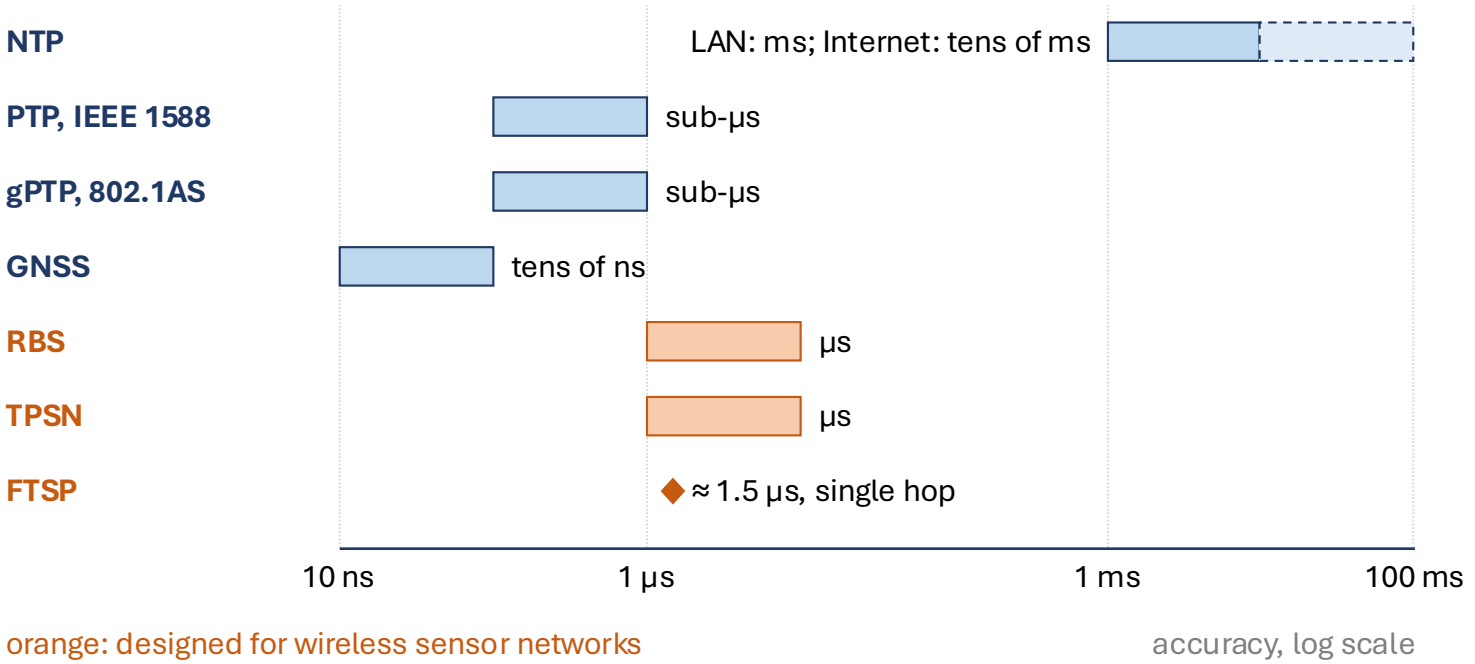
A message carries a timestamp across a path with several non-deterministic stages. Which of them a protocol can remove is what distinguishes one approach from another.

Stage	Determinism	Removed by
send time: application and OS scheduling	highly variable, milliseconds	timestamping at the MAC layer
access time: waiting for the medium	the most variable of all	timestamping at the MAC layer
transmission time	deterministic once the length is known	—
propagation time	negligible at these distances	—
reception and receive time	variable, OS dependent	hardware timestamping, or receiver-to-receiver comparison

This decomposition is why protocols differ by orders of magnitude in accuracy while all exchanging roughly the same messages. The accuracy is set by **where in the stack the timestamp is taken**, not by the algorithm.

Synchronisation protocols

Protocols exchange similar messages but differ by orders of magnitude: what decides is **where in the stack the timestamp is taken**.



where the error comes from

MAC

- send: application, OS
  ms, highly variable
- access: waiting for the medium
  the most variable

- transmission
  deterministic
- propagation
  negligible
- reception, receive time
  variable, OS dependent

A body sensor network has no GNSS indoors, no wired network and little energy: hence RBS, TPSN and FTSP. **MAC-layer timestamps** remove send and access time; RBS removes the sender.

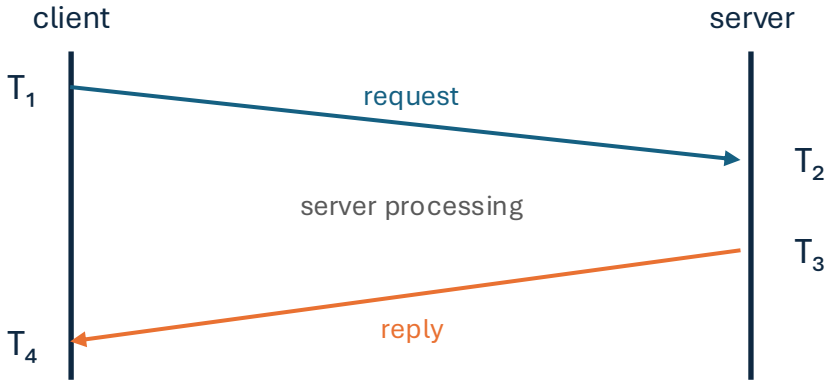
[30] Elson et al. 2002. Ganeriwal et al. 2003. Maróti et al. 2004, (Mica2 motes).

NTP in detail

Four timestamps are collected: departure from the client, arrival at the server, departure from the server, arrival back at the client.

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}, \delta = (T_4 - T_1) - (T_3 - T_2)$$

- θ is the offset estimate, δ the round-trip delay
- the estimate assumes the path is **symmetric**: that the two directions take the same time
- an asymmetry A produces an offset error of A/2, and it is **not observable** from the exchange: no number of repetitions removes it
- the local clock is then steered by a control loop rather than overwritten, so that time remains monotonic

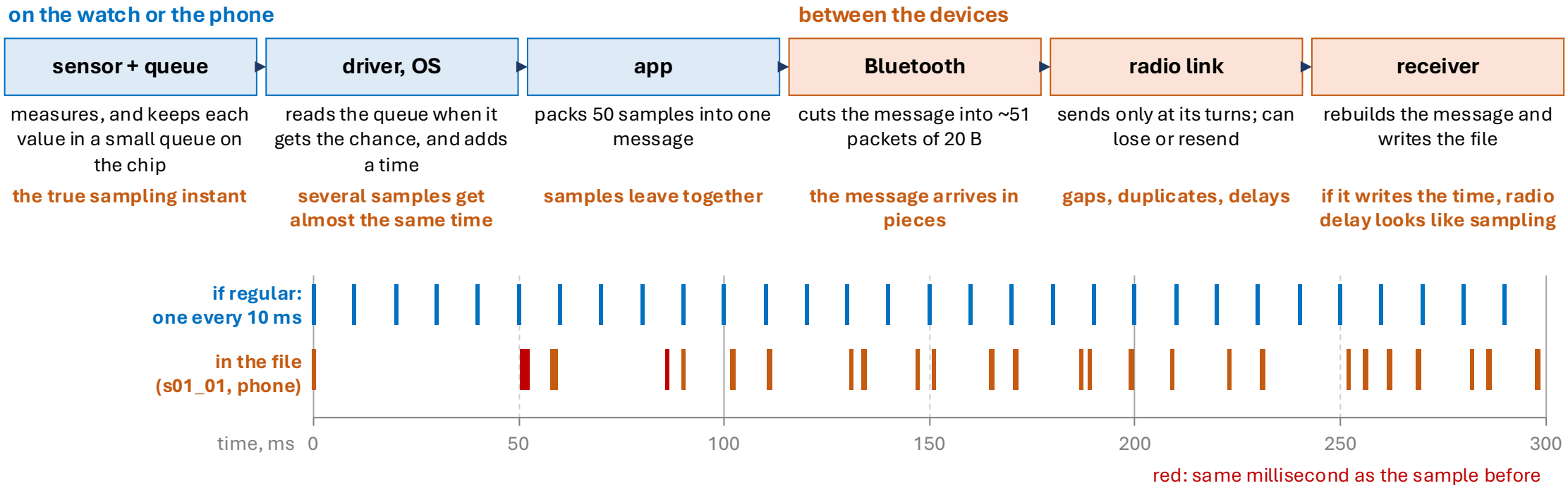


The four-timestamp exchange.

On a general-purpose operating system the wall clock can also be stepped by the synchronisation daemon. Samples must therefore be timestamped with a **monotonic** clock, and only the correspondence to wall time recorded separately.

From the sensor to the file

Between the sensor and the CSV file a sample goes through six steps. Each step can **delay** it, **group** it with others, or **drop** it, and the timestamp shows it.



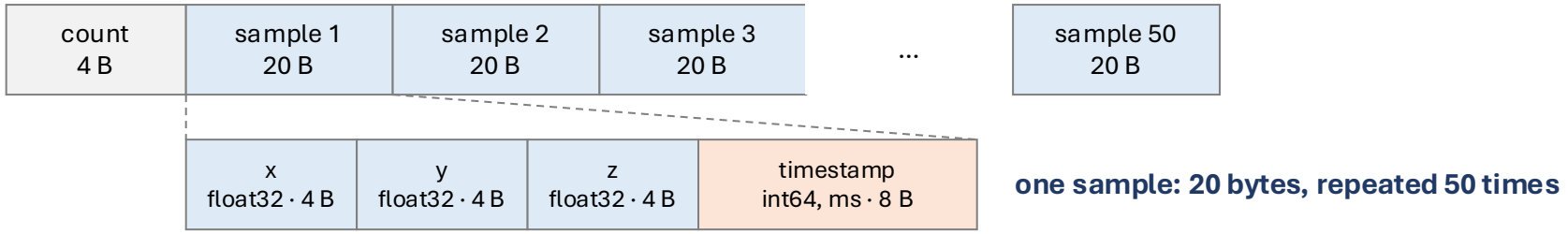
Stamp the time **as close to the sensor as possible**: every later step adds a delay that changes from sample to sample.

[21] STMicroelectronics, LSM6DSO datasheet (FIFO); [22] Android SensorManager documentation; Bluetooth Core Specification; WearOSSensors (GeoTecINIT); [4] Matey-Sanz, Casteleyn, Granell, Data in Brief 51, 2023; Zenodo 8398688 (s01_01, phone).

One batch on the wire

The watch does not send samples one by one: every 0.5 s it packs **50 samples** into one message, one message for the accelerometer and one for the gyroscope.

one message: 4 + 50 × 20 = 1 004 bytes



one sample: 20 bytes, repeated 50 times

what is sent	bytes per second, both sensors	bit rate
as recorded: float32 + a timestamp per sample	2 messages × 2 per s × 1 004 B = 4 016 B	**32.1 kbit/s**
without per-sample timestamps (start time + fixed rate)	100 per s × 2 sensors × 12 B = 2 400 B	**19.2 kbit/s**
...and int16 values instead of float32	100 per s × 2 sensors × 6 B = 1 200 B	**9.6 kbit/s**

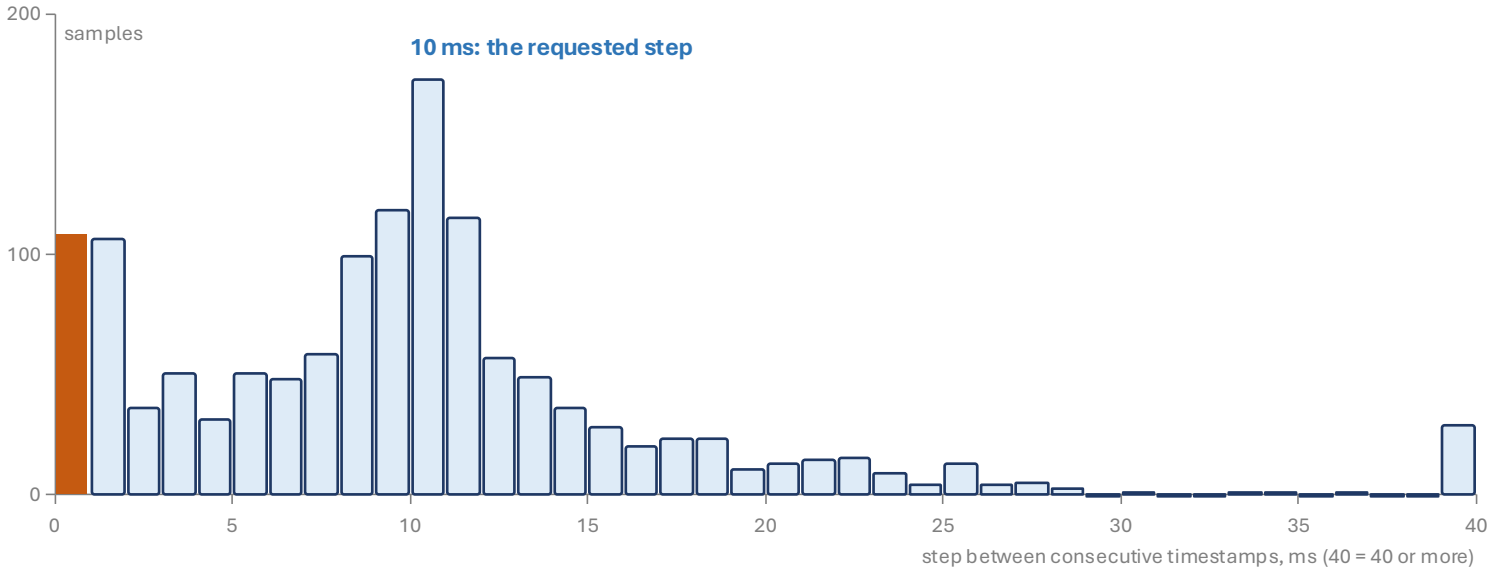
8 of every 20 bytes are timestamps. They cost bandwidth, but they are what shows late, grouped or lost samples: with only a start time and a fixed rate, those problems would be **invisible**.

WearOSSensors library (GeoTecINIT, Apache 2.0), TriAxialRecordCallback.encodeRecords; [3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §II-B1.

Timestamps of a real trial

The phone asked for 100 samples per second; this trial has 1351 samples in 13.2 s, 103 per second.
The steps between consecutive timestamps are not regular: many are 0 ms, some are much longer.

Below: how many times each step occurs. At 100 samples per second every step should be 10 ms.



0 ms
108 steps (8 %): the sample has the same timestamp as the one before. The two samples reached the app together, not 10 ms apart.

10 ms
The most frequent step, but only 173 times (13 %). Between 9 and 11 ms: 406 steps, less than one in three.

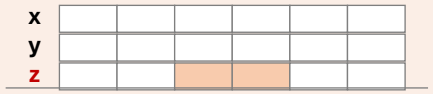
why
The sensor keeps its samples in a small queue on the chip. The phone's operating system passes them to the app only when it gives the app its turn to run, often several at once. The timestamp records when a sample reached the app, not when it was measured.

108 of 1351 timestamps (8%) repeat the previous one: samples reach the application in groups, through buffers of the operating system. For this reason every trial is resampled on a regular 50 Hz grid before analysis.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone); Android sensor framework: the requested rate is a hint, not a guarantee.

Loss, gaps and duplicates

The radio and the operating system leave four kinds of marks in the timestamps. Each one has a fix, and each fix has a **side effect**.

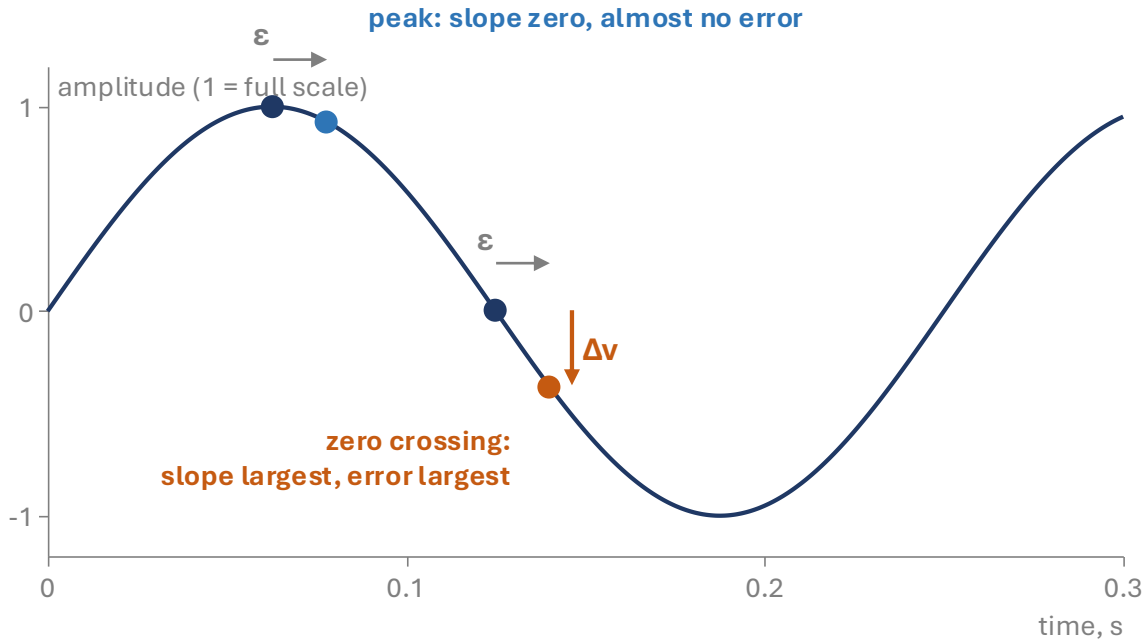
	a long gap	the same time twice	samples in bursts	a missing value
what you see				
why	radio packets lost, or the OS paused the app	samples arrived together and got the arrival time	the sensor queue is read all at once	an incomplete packet, or one sensor was off
what to do	mark it; short: interpolate; long: split the recording	keep their order; if you spread the times, say so	use the time of measurement, not of arrival	drop those windows, or fill them and say so
side effect	a window across the gap joins two different moments	the steps between samples are no longer the real ones	system delays look like irregular sampling	filled values are guesses, but a model treats them as measurements
in our data	**s01_01: longest step 64 ms, expected 10**	**s01_01: 108 of 1350 steps are 0 ms**	**s01_01: see «From the sensor to the file»**	**s01_01: 0 missing values**

After resampling to a regular grid none of these is visible any more: **look first**, and write down what you found and what you did.

[24] Zhang et al. (2024); [4] Matey-Sanz, Casteleyn, Granell, Data in Brief 51, 2023; Zenodo 8398688 (s01_01, phone).

Sampling-time error and amplitude error

If a sample is taken at t + **ε** instead of t, it reads the right signal at the wrong instant. The value is off by about **slope × ε**: Δv ≈ (dv/dt) · ε.



a 4 Hz sinusoid; ε = 15 ms, exaggerated so that it shows

how large: error ≈ 2π · f · σ_t of the amplitude

signal	σ_t	error	SNR
20 Hz: top of human movement	1 ms	12.6 %	18 dB
20 Hz: top of human movement	0.1 ms	1.3 %	38 dB
1.77 Hz: the steps of trial s01_01	5 ms	5.6 %	25 dB
1.77 Hz: the steps of trial s01_01	1 ms	1.1 %	39 dB

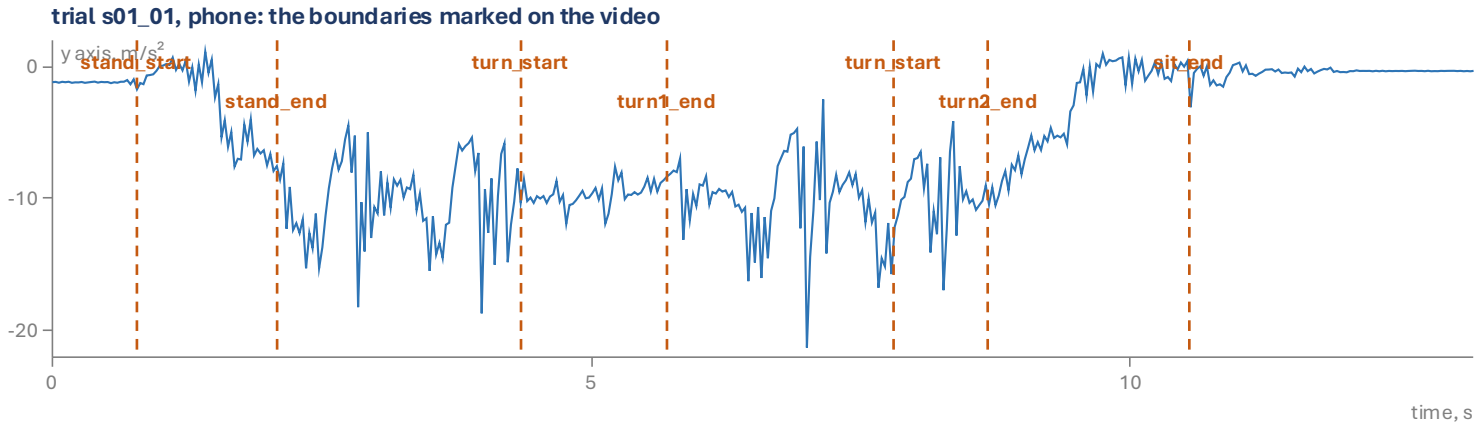
In the dataset files the irregular steps come mostly from when the system delivers the samples: the true σ_t is not written in the files, so 5 ms is an example.

The error is **largest where the signal changes fastest** and grows with **f · σ_t**: for the steps of the TUG a few milliseconds cost a few per cent; for a fast impact, much more.

First-order model Δv ≈ (dv/dt)·ε; [25] Kester, Analog Devices tutorial MT-007, 2009; [4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01).

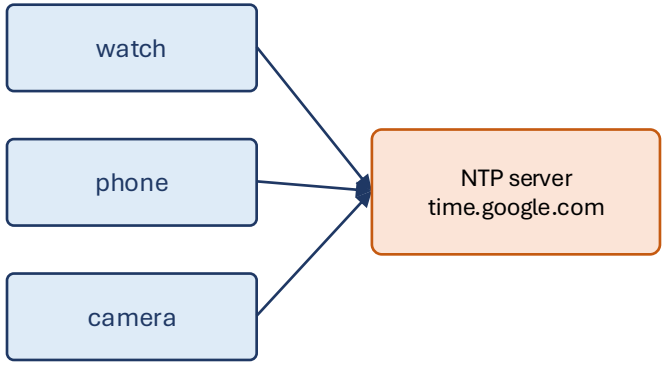
Labels and clocks

Every sample needs its phase. A researcher watched the video of each trial and marked the boundaries between phases, by rules written in advance; phone, watch and camera had synchronised their clocks with NTP before each session.



boundary	seen on the video when
stand_start	the back leaves the backrest
stand_end	upright, a foot rises for the first step
turn_start	shoulders or hips start to rotate

boundary	seen on the video when
turn1_end	facing the chair, a foot rises
turn2_end	knees bend, trunk leans forward: sitting starts
sit_end	the back touches the backrest



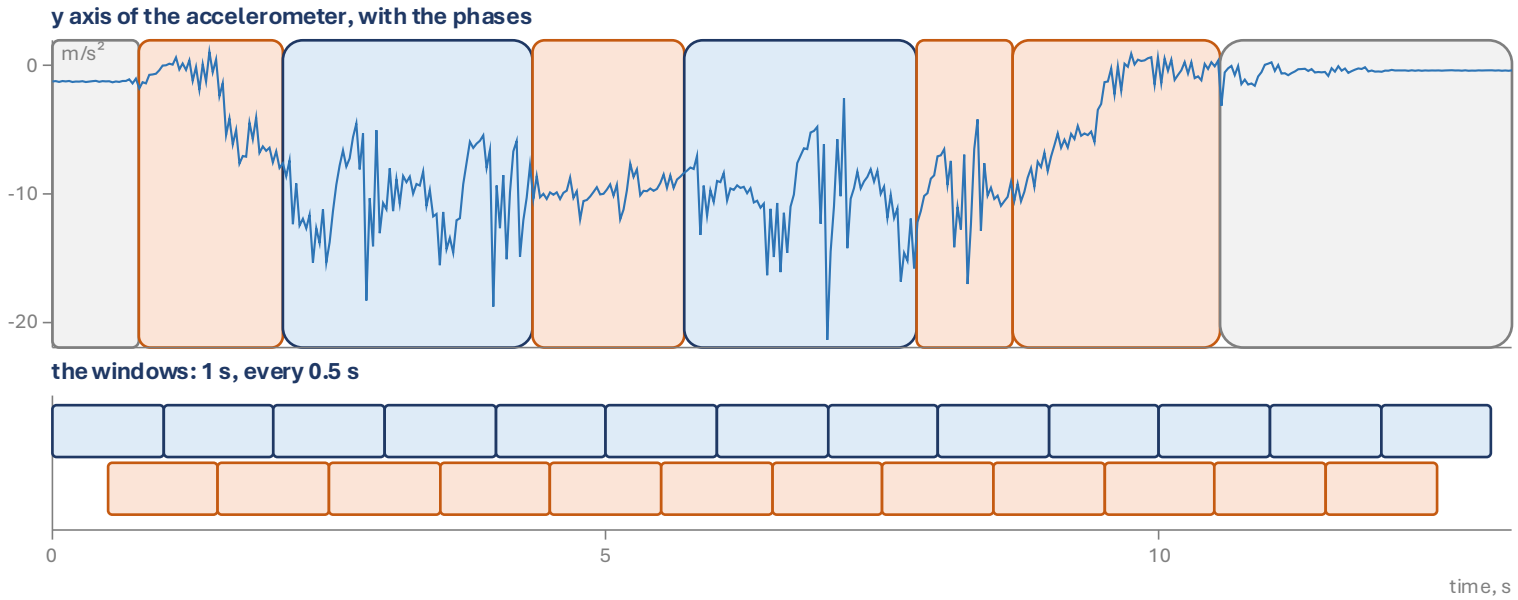
three clocks, synchronised before each session: the labels come from the camera, the samples from the phone and the watch

An offset between the camera's clock and the sensors' clock moves every boundary by the same amount.
The windows that cross a boundary are the ones whose label is uncertain.

[3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024, §II-C3 (NTP to time.google.com) and §II-C5 (boundaries); [4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone).

Cutting a trial into windows

A model takes inputs of fixed size, so the stream is cut into windows: here 50 samples (1 s at 50 Hz), one every 25 samples (0.5 s), half overlapping. Trial s01_01 gives 25 windows.



window
50 samples × 6 channels: the input of the model

hop
25 samples: each sample falls in two windows

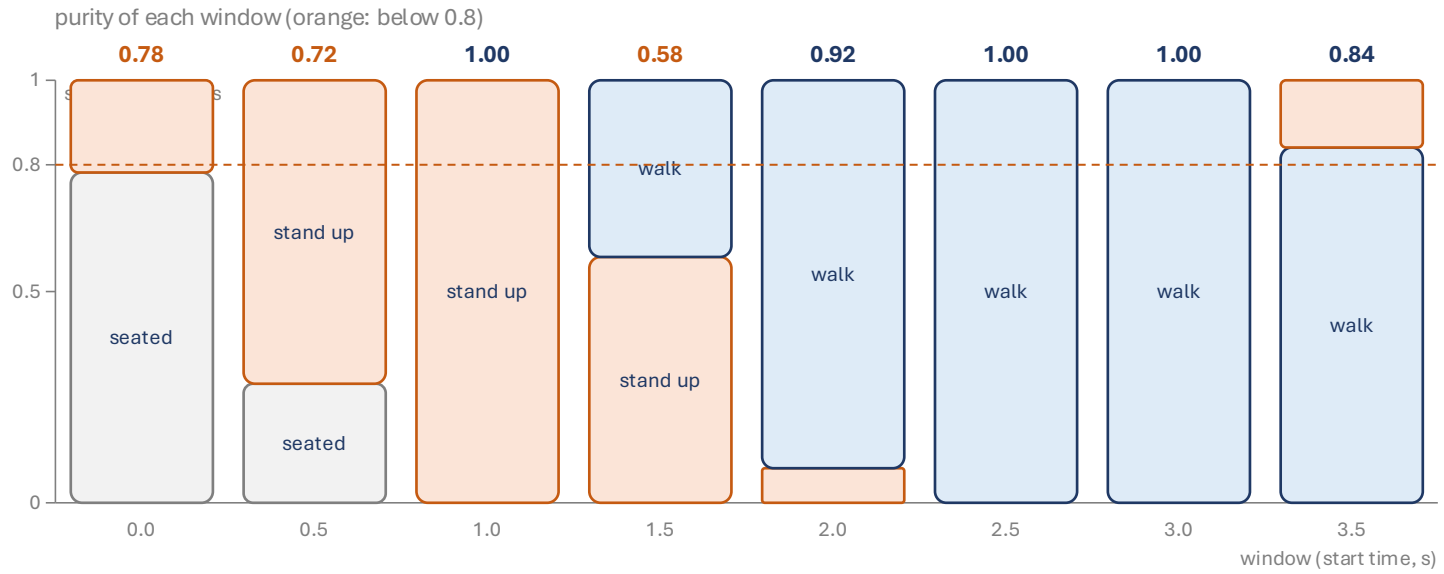
at the edges
the last samples that do not fill a window are dropped

Window length and hop are design choices: a longer window sees more of the movement but mixes phases and delays the decision; a shorter hop gives more windows, which overlap more and are more alike.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, phone, y axis, resampled to 50 Hz).

Which label does a window get?

A window that crosses a phase boundary contains two labels. It takes the majority label; its purity is the share of samples with that label. Here the first windows of s01_01, around standing up.



majority label
the label with most samples in the window

purity
the share of that label: 1 = one phase only

threshold
drop windows with purity below 0.8, or keep them and accept noisy labels

Windows with low purity are the uncertain ones: half seated, half standing up. A common choice is to drop windows below a purity threshold, for example 0.8: fewer but cleaner examples, and fewer examples of the short phases.

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01).

Windows per phase

Counting the windows of trial s01_01 by majority label: all of them, and those with purity of at least 0.8. Short phases give few windows, and they lose most of them to the threshold.



in total
25 windows; 17 with purity ≥ 0.8

the short phases
a 1 s window rarely fits inside a transition of about 1 s

on the whole dataset
the same count, over all the trials and per device

The classes are unbalanced before any model: walking dominates, standing up and sitting down are rare and lose most windows at the boundaries. Over the whole dataset the picture is the same: a fair metric must weigh every class equally (macro-F1).

[4] Matey-Sanz et al., Data in Brief 51, 2023, (trial s01_01, windows of 1 s, hop 0.5 s, at 50 Hz).

References cited as [n]

[1] Podsiadlo and Richardson, J. Am. Geriatr. Soc. 39(2), 1991
[2] Shumway-Cook, Brauer and Woollacott, Phys. Ther. 80(9), 2000
[3] Matey-Sanz, Casteleyn, Granell, González-Pérez, IEEE J. Biomed. Health Inform. 28(11), 2024
[4] Matey-Sanz et al., Data in Brief 51, 2023
[5] Hajar, Al-Kadri and Kalutarage, Computers & Security 104, 2021
[6] Chen et al., «Body area networks, a survey», Mobile Netw. Appl. 16(2), 2011
[7] IEEE Std 802.15.6-2012
[8] Microchip ATmega328P and Allwinner H3 datasheets
[9] Ojo et al., IEEE Access 6, 2018
[10] RAM from the datasheets
[11] tiers after Ojo et al., IEEE Access 6, 2018
[12] RFC 7228
[13] Farahani et al., IEEE Internet of Things Magazine, 2025
[14] Tanenbaum and Bos, Modern Operating Systems, 4th ed. (2015)
[15] Farahani et al., IEEE IoT Magazine, 2025
[16] RIOT documentation
[17] Buttazzo, Hard Real-Time Computing Systems, 3rd ed. (2011)
[18] FreeRTOS and Zephyr documentation
[19] Contiki-NG documentation
[20] project documentation of each OS
[21] NXP UM10204
[22] STMicroelectronics, LSM6DSO datasheet
[23] Zephyr sensor subsystem documentation
[24] Android SensorManager documentation
[25] Bluetooth Core Specification (ATT MTU, connection interval). IEEE 802.15.4
[26] Zhang et al. (2024)
[27] «Riconoscere un gesto dall'EMG di un bracciale», documento di studio, v2, 2026, cap. 3
[28] Epson FC-135 crystal datasheet (32.768 kHz, ±20 ppm at 25 °C)
[29] nRF52 product specification
[30] Elson et al. 2002. Ganeriwal et al. 2003. Maróti et al. 2004

References

Source	Title
JCGM 100:2008	Evaluation of measurement data — Guide to the expression of uncertainty in measurement (GUM).
JCGM 101:2008	Supplement 1 to the GUM — Propagation of distributions using a Monte Carlo method.
JCGM 200:2012	International vocabulary of metrology — Basic and general concepts (VIM), 3rd ed.
Titterton and Weston, 2004	Strapdown Inertial Navigation Technology, 2nd ed., IET/AIAA.
El-Sheimy, Hou and Niu, 2008	Analysis and Modeling of Inertial Sensors Using Allan Variance. IEEE Trans. Instrum. Meas. 57(1), 140–149.
Mahony, Hamel and Pflimlin, 2008	Nonlinear Complementary Filters on the Special Orthogonal Group. IEEE Trans. Automatic Control 53(5), 1203–1218.
Harris, 1978	On the use of windows for harmonic analysis with the discrete Fourier transform. Proc. IEEE 66(1), 51–83.
Cooley and Tukey, 1965	An algorithm for the machine calculation of complex Fourier series. Math. Comput. 19(90), 297–301.
Mills et al., 2010	Network Time Protocol Version 4: Protocol and Algorithms Specification. IETF RFC 5905.